

**LAPORAN PRAKTIKUM PEMBELAJARAN MESIN: ANALISIS KINERJA K-
NEAREST NEIGHBORS (KNN) DAN SUPPORT VECTOR MACHINE (SVM)
PENGOLAHAN CITRA DIGITAL**



Dosen Pengampu:
Dr. Yasdinul Huda, S.Pd., M.T.

Oleh:
Muhammad Zahran
24343077

**PROGRAM STUDI INFORMATIKA
DEPARTEMEN TEKNIK ELEKTRONIKA
FAKULTAS TEKNIK
UNIVERSITAS NEGERI PADANG
2026**

A. Pendahuluan

Klasifikasi merupakan salah satu tugas fundamental dalam pembelajaran mesin yang bertujuan untuk memetakan data ke dalam kategori tertentu berdasarkan fitur-fiturnya. Laporan ini membahas implementasi dan evaluasi dua algoritma klasifikasi populer, yaitu *K-Nearest Neighbors* (KNN) dan *Support Vector Machine* (SVM). Praktikum ini terbagi menjadi tiga eksperimen utama: pengenalan digit menggunakan KNN, analisis perbandingan kernel SVM pada dataset sintetis, dan komparasi komprehensif antara KNN dan SVM menggunakan dataset standar. Tujuannya adalah untuk memahami karakteristik, kelebihan, serta kepekaan masing-masing model terhadap parameter dan distribusi data.

B. Metodologi

Penelitian ini menggunakan pustaka scikit-learn dalam bahasa pemrograman Python. Evaluasi model dilakukan menggunakan teknik validasi silang (*cross-validation*) dengan 5 lipatan (5-fold) untuk memastikan keandalan metrik kinerja, serta pembagian data latih dan uji dengan proporsi 70:30. Adapun tiga dataset yang digunakan meliputi:

- **Dataset MNIST:** Kumpulan data citra piksel beresolusi 28x28 yang merepresentasikan digit angka (0-9).
- **Dataset Buah Sintetis:** Data buatan berskala 20 dimensi yang merepresentasikan tiga kelas buah (Apel, Pisang, Jeruk) dengan karakteristik fitur yang tumpang tindih. Data ini direduksi menggunakan *Principal Component Analysis* (PCA) menjadi 2 dimensi untuk keperluan visualisasi.
- **Dataset Iris:** Kumpulan data standar botani bunga Iris yang digunakan untuk tolok ukur perbandingan akhir

Untuk model SVM, seluruh fitur numerik distandardisasi terlebih dahulu menggunakan *StandardScaler* agar variansi data berada pada skala yang seragam, sehingga proses pencarian *hyperplane* optimal tidak bias terhadap fitur dengan rentang nilai yang besar.

C. Hasil dan Pembahasan

1. Eksperimen 1: Pengenalan Digit dengan KNN

Pada eksperimen pertama, algoritma KNN dievaluasi dalam mengenali pola citra digit. Eksperimen dilakukan dengan menguji berbagai nilai tetangga terdekat, yaitu $k \in \{1, 3, 5, 7, 9, 11\}$.

Berdasarkan alur program, evaluasi menunjukkan keseimbangan antara metrik akurasi validasi silang dan akurasi data uji. Penentuan nilai k terbaik didasarkan pada skor validasi silang tertinggi untuk mencegah fenomena *overfitting* yang rentan terjadi pada $k = 1$. Model dievaluasi lebih lanjut menggunakan *Confusion Matrix* untuk melihat persebaran prediksi kelas yang benar dan salah. Algoritma program juga secara spesifik mengekstraksi dan memvisualisasikan data yang salah diklasifikasikan (*misclassifications*). Hal ini secara akademik sangat berguna untuk menganalisis apakah kesalahan prediksi disebabkan oleh kemiripan visual antar digit (misalnya, angka 3 yang ditulis menyerupai angka 8).

2. Eksperimen 2: Analisis Kernel SVM pada Data Sintetis

Eksperimen kedua menyoroti fleksibilitas SVM dalam menangani pemisahan kelas melalui trik kernel. Pada dataset sintetik klasifikasi buah berdimensi tinggi yang telah melalui reduksi PCA, empat jenis kernel dievaluasi:

- **Linear:** Cocok untuk data yang dapat dipisahkan secara garis lurus.
- **Polynomial (Derajat 3):** Memproyeksikan data ke ruang berdimensi lebih tinggi menggunakan fungsi polinomial.
- **RBF (Radial Basis Function):** Sangat tangguh untuk memetakan distribusi data yang kompleks dan tidak linear.
- **Sigmoid:** Menggunakan fungsi aktivasi serupa jaringan saraf tiruan.

Hasil eksperimen memvisualisasikan batasan keputusan (*decision boundary*) dari masing-masing kernel di ruang dua dimensi PCA. Evaluasi kinerja terbaik diukur melalui nilai akurasi dan Kurva ROC (*Receiver Operating Characteristic*). Kurva ROC yang diimplementasikan menggunakan strategi *One-vs-Rest* (OvR) memberikan metrik *Area Under Curve* (AUC), yang secara komprehensif mengukur kemampuan model dalam membedakan masing-masing kelas buah (Apel, Pisang, Jeruk) terhadap kelas lainnya.

3. Eksperimen 3: Perbandingan Komprehensif KNN vs SVM

Bagian akhir praktikum merupakan komparasi langsung antara model KNN ($k = 5$), SVM dengan kernel Linear, dan SVM dengan kernel RBF pada dataset Iris.

Pendekatan evaluasi pada tahap ini jauh lebih ketat, tidak hanya mengandalkan metrik akurasi, tetapi juga menyertakan *Precision* (ketepatan prediksi positif), *Recall* (kemampuan menemukan semua sampel positif), dan *F1-*

Score (rata-rata harmonik dari Presisi dan Recall). Berdasarkan analisis arsitektur algoritma, SVM (khususnya RBF) umumnya menunjukkan keunggulan pada generalisasi data yang memiliki margin kelas yang sempit atau *outlier*, sedangkan KNN unggul dalam kesederhanaan komputasi pada saat pelatihan (*training*) namun memerlukan memori tinggi saat pengujian karena harus menghitung jarak ke seluruh titik data latih.

D. Kesimpulan

Praktikum ini memberikan wawasan empiris mengenai perilaku KNN dan SVM:

1. **Kinerja KNN** sangat bergantung pada pemilihan jumlah tetangga terdekat (k) dan cukup rentan terhadap *noise* visual pada data seperti pengenalan citra beresolusi rendah.
2. **Kinerja SVM** sangat dipengaruhi oleh pemilihan tipe kernel. Pra-pemrosesan data seperti standardisasi mutlak diperlukan untuk SVM agar mencapai metrik AUC dan akurasi yang optimal.
3. Secara komparatif, SVM dengan kernel non-linear (seperti RBF) cenderung lebih superior dalam membentuk batasan keputusan pada dataset yang kompleks dibandingkan pendekatan jarak spasial konvensional yang digunakan oleh KNN. Evaluasi menggunakan metrik gabungan (Akurasi, Presisi, Recall, dan F1-Score) membuktikan pentingnya pemilihan algoritma yang disesuaikan dengan struktur intrinsik dataset.

E. Lampiran Program dan Output

1. Kode Program

```
109
110
111 # Import necessary libraries
112 import numpy as np
113 import matplotlib.pyplot as plt
114 from sklearn.datasets import load_digits
115 from sklearn.neighbors import KNeighborsClassifier
116 from sklearn.model_selection import train_test_split, cross_val_score
117 from sklearn.metrics import classification_report, confusion_matrix
118 import os
119
120 # Define the main function
121 def main():
122     # Load the dataset
123     digits = load_digits()
124     print(f"Dataset shape: {digits.data.shape}")
125     print(f"Number of classes: {len(np.unique(digits.target))}")
126
127     # Split the data into training and testing sets
128     X_train, X_test, y_train, y_test = train_test_split(
129         digits.data, digits.target, test_size=0.2, random_state=42, stratify=digits.target)
130
131     # Create a KNeighborsClassifier object
132     knn = KNeighborsClassifier(n_neighbors=5)
133
134     # Train the model
135     knn.fit(X_train, y_train)
136
137     # Predict the test set
138     y_pred = knn.predict(X_test)
139
140     # Calculate the accuracy
141     accuracy = knn.score(X_test, y_test)
142     print(f"Accuracy: {accuracy}")
143
144     # Confusion matrix
145     cm = confusion_matrix(y_test, y_pred)
146     plt.figure(figsize=(10, 8))
147     cm.imshow(cm, cmap='gray')
148     plt.title('Confusion Matrix')
149     plt.xlabel('Predicted')
150     plt.ylabel('Actual')
151     plt.show()
152
153     # Classification report
154     print(classification_report(y_test, y_pred))
155
156     # Cross-validation
157     cv_scores = cross_val_score(knn, X_train, y_train, cv=5, scoring='accuracy')
158     print(f"CV Accuracy: {cv_scores.mean():.4f}, Test Accuracy: {accuracy:.4f}")
159
160     # Feature importance
161     # Note: KNeighborsClassifier does not have a built-in feature importance method.
162     # We can use a wrapper or a different model for this.
163
164     # Save the model
165     joblib.dump(knn, 'knn_model.pkl')
166
167     # Load the model
168     knn_loaded = joblib.load('knn_model.pkl')
169
170     # Predict using the loaded model
171     y_pred_loaded = knn_loaded.predict(X_test)
172
173     # Compare the results
174     print(f"Original Accuracy: {accuracy:.4f}, Loaded Accuracy: {knn_loaded.score(X_test, y_test):.4f}")
175
176     # Plot the decision boundary
177     # Note: KNeighborsClassifier does not have a built-in decision boundary method.
178     # We can use a wrapper or a different model for this.
179
180     # Main execution
181     main()
182
183 # Run the main function
184 if __name__ == '__main__':
185     main()
186
187 # End of the program
188
```

```

215 # Compute ROC curve and auc area for each class
216 plt.figure(figsize=(10, 8))
217 for i in range(len(fruit_names)):
218     fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_score[:, i])
219     roc_auc = auc(fpr, tpr)
220     plt.plot(fpr, tpr, label=
221             f'Label-{fruit_names[i]} (AUC = {roc_auc:.4f})')
222
223 plt.plot([0, 1], [0, 1], 'k--', label=
224         'Random Guess')
225 plt.xlabel('False Positive Rate')
226 plt.ylabel('True Positive Rate')
227 plt.title('ROC Curve - One with (best_kernel.upper()) Kernel')
228 plt.legend(loc='lower right')
229 plt.grid(True, alpha=0.5)
230 plt.show()
231
232 # Feature importance for linear kernel
233 if best_kernel == 'linear':
234     coef = results['linear']['model'].coef_
235     plt.figure(figsize=(10, 8))
236     for i in range(coef.shape[0]):
237         plt.bar(range(coef.shape[1]), coef[i], alpha=0.8,
238                 label=f'Class {fruit_names[i]} vs Test')
239         plt.xlabel('Feature Index')
240         plt.ylabel('Coefficient Value')
241         plt.title(f'Feature Importance (linear kernel)')
242         plt.legend()
243         plt.grid(True, alpha=0.5, axis='y')
244         plt.show()
245
246 return results, fruit_names
247
248 # Balance dataset
249 use_results, fruit_names = prepare_use_fruits()
250
251 def compare_knn_svm():
252     print("\n===== KNN vs SVM")
253     print("\n" * 5)
254
255 # Load iris dataset for comparison
256 from sklearn.datasets import load_iris
257
258 iris = load_iris()
259 X = iris.data
260 y = iris.target
261
262 X_train, X_test, y_train, y_test = train_test_split(
263     X, y, test_size=0.3, random_state=0, stratify=y)
264
265 scaler = StandardScaler()
266 X_train_scaled = scaler.fit_transform(X_train)
267 X_test_scaled = scaler.transform(X_test)
268
269 # Initialize models
270 knn = KNeighborsClassifier(n_neighbors=5)
271 svm_linear = svm.SVC(kernel='linear', C=1.0, random_state=0)
272 svm_rbf = svm.SVC(kernel='rbf', C=1.0, gamma='scale', random_state=0)
273
274 models = {
275     'knn (k=5)': knn,
276     'svm linear': svm_linear,
277     'svm rbf': svm_rbf
278 }
279
280 results = {}
281
282 for name, model in models.items():
283     # Train-validation
284     cv_scores = cross_val_score(model, X_train_scaled, y_train, cv=5)
285
286     # Training and testing
287     model.fit(X_train_scaled, y_train)
288     y_pred = model.predict(X_test_scaled)
289
290     # Calculate metrics
291     from sklearn.metrics import precision_score, recall_score, f1_score
292
293     accuracy = model.score(X_test_scaled, y_test)
294     precision = precision_score(y_test, y_pred, average='weighted')
295     recall = recall_score(y_test, y_pred, average='weighted')
296     f1 = f1_score(y_test, y_pred, average='weighted')
297
298     results[name] = {
299         'cv_accuracy': cv_scores.mean(),
300         'test_accuracy': accuracy,
301         'precision': precision,
302         'recall': recall,
303         'f1': f1,
304         'model': model
305     }
306
307 # Display comparison
308 print("\nModel\t\tCV Acc\t\tTest Acc\t\tPrecision\t\tRecall\t\tF1-Score")
309 print("\n" * 5)
310 for name, result in results.items():
311     print(f"{name}\t\t{result['cv_accuracy']:.4f}\t\t{result['test_accuracy']:.4f}\t\t"
312           f"{result['precision']:.4f}\t\t{result['recall']:.4f}\t\t{result['f1']:.4f}")
313
314 # Visual comparison
315 metrics = ['CV Accuracy', 'Test Accuracy', 'Precision', 'Recall', 'F1-Score']
316 model_names = list(models.keys())
317
318 fig, axes = plt.subplots(5, 3, figsize=(15, 14))
319 axes = axes.ravel()
320
321 for idx, metric in enumerate(metrics):
322     values = [results[name][metric] for name in model_names]
323
324     bars = axes[idx].bar(model_names, values, color='teal', edgecolor='lightgray',
325                          label=f'{metric}', title='')
326     axes[idx].set_title(metric.replace('_', ' ').title())
327     axes[idx].set_xlabel('Model')
328     axes[idx].set_ylabel('Value')
329
330     for bar, value in zip(bars, values):
331         axes[idx].text(bar.get_x() + bar.get_x() + bar.get_x() / 2, bar.get_height() + 0.02,
332                        f'{value:.4f}', ha='center', va='bottom')
333
334 # Hide empty subplot
335 axes[-1].axis('off')
336
337 plt.suptitle('KNN vs SVM Performance Comparison', fontsize=14)
338 plt.tight_layout()
339 plt.show()
340
341 return results
342
343 # Balance performance
344 compare_results = compare_knn_svm()

```

2. Screenshot Output





